NALAR

FINAL PROJECT

Movie Recommendation System

Content-Based Filtering · TF-IDF + Cosine Similarity + Bayesian · TMDB Dataset

Wa Ode Nisrina Sayyidah Hidayat

Python | Tableau | Streamlit

Nalar Bootcamp · 2026

NALAR

Wa Ode Nisrina Sayyidah Hidayat
Program Studi S1 Statistika - Universitas Gadjah Mada

linkedin.com/waodenisrina

github.com/waodenisrina-analytics

Repository Projek di Github

NALAR

🔍 Background & Permasalahan

❗ Masalah

- Jutaan film tersedia, pengguna kesulitan memilih
- Pilihan berlebih menyebabkan information overload
- Tidak ada panduan personal berbasis preferensi konten
- Platform streaming kehilangan engagement user

🎯 Konteks Industri

- Industri streaming global tumbuh pesat (~$500B by 2030)
- Netflix, Disney+, Prime Video bersaing ketat
- Sistem rekomendasi = fitur kunci retensi pengguna
- Content-based filtering: solusi tanpa butuh data historis user

💡 Mengapa Proyek Ini Penting?

Sistem rekomendasi berbasis konten memungkinkan personalisasi **tanpa riwayat interaksi user,** cocok untuk cold-start problem. Dengan memanfaatkan **overview, genre, dan keywords film**, sistem mampu menyarankan film relevan secara otomatis dan skalabel.

NALAR

🚀 Goal & Impact

🎯

Tujuan Utama

- Bangun sistem rekomendasi film berbasis konten
- Hasilkan Top-N (5-20) film yang relevan dari satu judul input
- Gabungkan TF-IDF similarity + Bayesian quality score

👤

Manfaat untuk User

- Temukan film serupa dengan cepat & akurat
- Filter berdasarkan genre favorit
- Akses sinopsis & metadata langsung di UI

💼

Manfaat Bisnis

- Tingkatkan watch-time & retensi pengguna
- Kurangi churn rate pada platform streaming
- Solusi scalable tanpa user history

Flow Project

NALAR

01

Data Understanding

TMDB Dataset
~1M films

02

Preprocessing and EDA

Cleaning· Filtering · Combined Features · EDA Histogram, scatter, bar chart

03

Feature Engineering

TF-IDF dan Cosine Similarity Scoring

04

Modelling and Evaluation

Weighted Score with Bayesian Average. Evaluation with Precision@K.

05

Deployment

Using github and Streamlit

Preprocessing, FE, Modelling, Evaluation -> Jupyter Notebook
EDA -> Tableau
Deployment -> Python, Github, Streamlit

NALAR

Implementation: Data Understanding & Preprocessing

Data Understanding

Dataset : TMDB Kaggle (The Movie Database)
Ukuran : **1.399.719 baris** dengan **24 kolom**
Kolom dipilih (6) :

Kolom	Alasan
Title	Identitas film, input & output sistem
Overview	Deskripsi naratif yang kaya makna semantik
Genres	Sinyal utama kemiripan selera penonton
Keywords	Menangkap nuansa konten yang tidak ada di overview
Vote_average	Untuk menghitung kualitas film (Weighted Score)
Vote_count	Filter film dengan popularitas rendah

Preprocessing

Ditemukan beberapa masalah :
- keywords kosong di **1.051.733** baris
- genres kosong di **611.413** baris
- overview kosong di **319.703** baris
- Ada **8.126** baris duplikat

Penanganan dan Filtering:
- Hapus baris dengan overview, genres, atau title kosong, mengisi baris kosong keywords dengan “ “
- Hapus film dengan vote_count < 100 dan vote_average < 1.0 untuk menghilangkan film yang sangat jarang ditonton dan menghilangkan data anomali
- Text cleaning (standardisasi text) : lowercasing, regex filtering (hapus tanda baca dan angka), whitespace normalization, separator handling (koma jadi spasi)
- Menambah fitur baru combined_features: menggabung semua kolom teks jadi 1 string Tunggal. Mengulang genres dan keywords 2x agar mendapat bobot lebih di TF-IDF.

📊 Implementation: Exploratory Data Analysis

NALAR

Visualisasi menggunakan Tableau

🔔 Distribusi Vote Average

Distribusi hampir mirip normal namun sedikit skew-negative (left-skewed)

📈 Scatter Vote Count dan Vote Average

Terbukti adanya Law of Large Number, film yang lebih populer, vote lebih stabil.

🏆 Top Genre dan Film Populer

Drama & Comedy mendominasi lintas usia & budaya. Film dengan fans solid cenderung populer.

Implementation: Feature Engineering

NALAR

TF-IDF Vectorization

TF-IDF merupakan metode pembobotan yang digunakan untuk mengukur tingkat kepentingan suatu kata dalam dokumen relatif terhadap keseluruhan korpus. **Memanfaatkan fitur *combined_feature*** yang telah dibuat sebelumnya untuk memberikan “skor kepentingan” untuk setipa kata.

- TF (Term Frequency): Melihat seberapa sering kata itu muncul dalam satu film.
- IDF (Inverse Document Frequency): Melihat sberapa langka kata itu di semua film.
- Skor akhir = TF × IDF -> kata-kata unik dan sering dalam satu film mendapat skor tinggi.

Implementasi menggunakan **TfidfVectorizer** (mengubah teks menjadi vector angka):

- max_features (15.000) -> Hanya ambil 15.000 kata paling informatif dari seluruh korpus
- ngram_range(1,2) -> Tangkap kata tunggal ("action") DAN frasa dua kata ("science fiction")
- min_df (2) -> Buang kata yang hanya muncul di 1 film karena terlalu spesifik, tidak berguna untuk perbandingan
- sublinear_tf (True) -> Kata yang muncul 20× tidak dianggap 20× lebih penting dari yang muncul 1× pakai log
- stop_words ("english") -> Hapus kata tak bermakna seperti "the", "a", "is", "in"

Persamaan TF

$$wf_{t,d} = 1 + \log(f_{t,d})$$

Persamaan IDF

$$idf(t) = \log \frac{n}{df(t)} + 1$$

Skor Akhir

$$W_{t,d} = wf_{t,d} \times idf(t)$$

NALAR

Implementation: Model & Scoring

Dalam tahap Modelling, digunakan pendekatan **Content-Based Filtering** yang menggabungkan dua metode utama: **Cosine Similarity** untuk mengukur kemiripan konten antar film, dan **Bayesian Weighted Score** untuk memastikan film yang direkomendasikan juga berkualitas.

Cosine Similarity

Setelah setiap film direpresentasikan sebagai vektor TF-IDF, kemiripan antar film diukur menggunakan Cosine Similarity, yaitu kosinus sudut antara dua vektor:

$$\text{similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|}$$

- Nilai: 0 (tidak mirip) → 1 (identik)
- Efisien: dihitung on-demand per query

Weighted Score dengan Bayesian Average

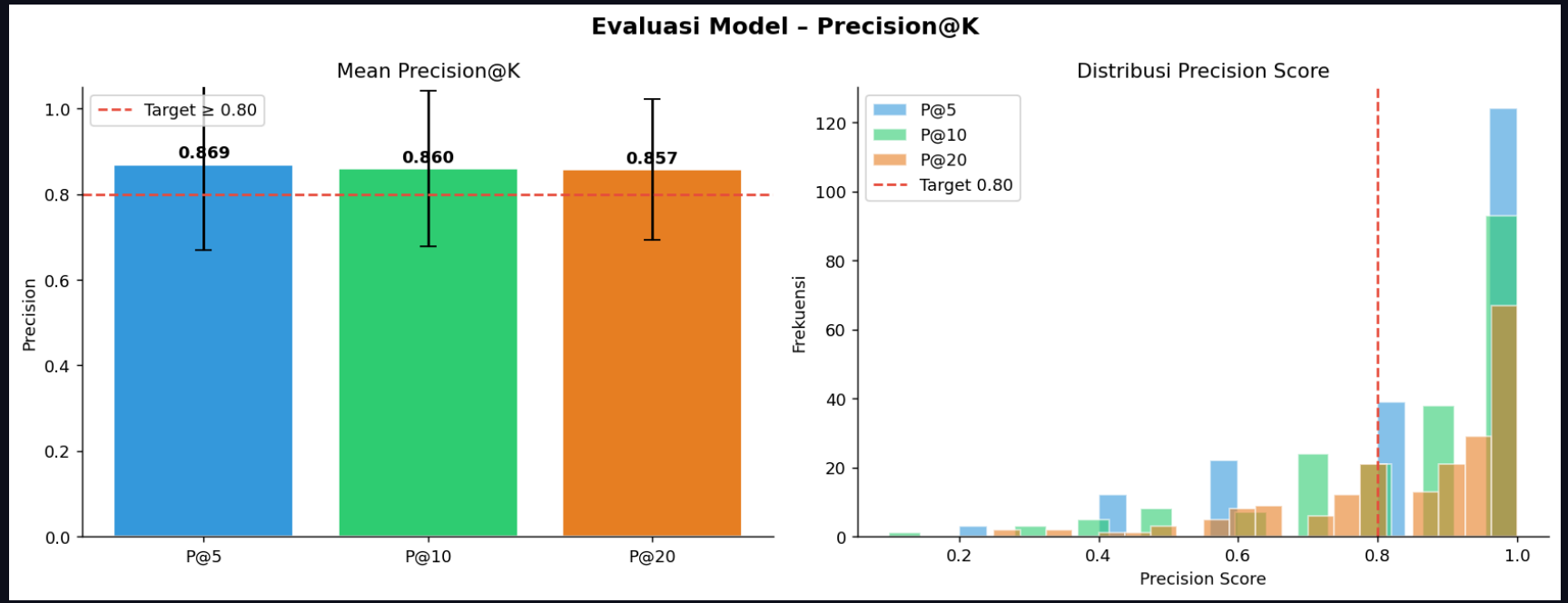
Bayesian Weighted Score menyelesaikan masalah pertimbangan dan kualitas film agar adil:

$$WS = \frac{v}{v+m}R + \frac{m}{v+m}C$$

di mana R = rata-rata rating film tersebut, v = jumlah vote film tersebut, C = rata-rata rating global seluruh dataset (= 6.513), m = ambang minimum vote dari persentil ke-60 (= 418 vote). Semakin besar v, semakin besar porsi R yang digunakan, artinya rating asli film semakin dipercaya.

Hasil & Evaluasi

NALAR



NALAR

📈 Hasil & Evaluasi

18,139×15K	0.8690	0.8600	0.8572
Dimensi TF-IDF	Precision@5	Precision@10	Precision@20

✅ Interpretasi Precision@K

- Precision@5 = 0.8690 -> dari Top-5 rekomendasi, rata-rata >86.9% memiliki genre yang overlap dengan film query.
- Precision@10 = 0.8600 -> performa sedikit turun karena bucket lebih besar, namun tetap di atas target 0.80.
- Precision@20 = 0.8572 -> masih kuat meski K semakin besar.

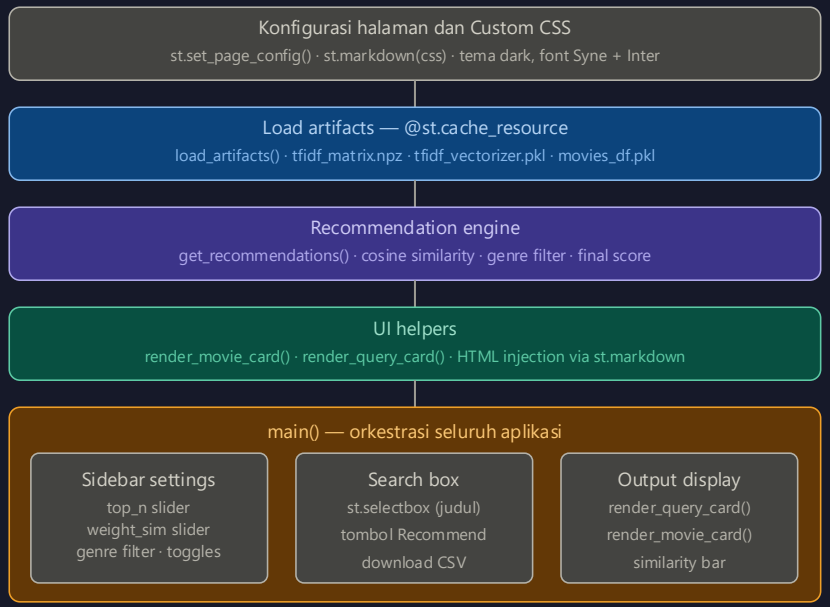
Model berhasil mencapai/melampaui target Precision ≥ 0.80, memvalidasi bahwa TF-IDF + Cosine Similarity efektif untuk content-based filtering.

Aplikasi Streamlit

🎬 CineMatch App

NALAR

CineMatch adalah aplikasi web berbasis Streamlit yang menjadi antarmuka (front-end) dari Movie Recommendation System yang telah dibangun di notebook. Tujuannya sederhana: pengguna memilih satu judul film, lalu sistem menampilkan daftar film yang paling mirip secara konten sekaligus berkualitas.




```
[train.py dijalankan sekali]
  └── Hasilkan artifacts/ (tfidf_matrix, vectorizer, movies_df)

[User buka app: streamlit run app.py]
  └── load_artifacts() -> muat 3 file yang dihasilkan train.py
  └── Render sidebar + search box

[User pilih film -> klik Recommend]
  └── get_recommendations() dipanggil
      ├── Cari query_idx dari title_to_idx
      ├── Hitung cosine_similarity (1 baris vs 18.139 baris)
      ├── Terapkan genre filter jika dinyalakan
      ├── Hitung final_score = 70% sim + 30% ws_norm (adjustable)
      └── Sort descending → ambil Top-N

[Hasil ditampilkan]
  └── render_query_card() → tampilkan info film yang dicari
  └── render_movie_card() × N → tampilkan kartu rekomendasi
  └── st.download_button() → opsi export CSV
```

🎬 Aplikasi Streamlit CineMatch

NALAR

🎬 CineMatch App

1

Pilih Film

Cari & pilih film dari dropdown (18K judul tersedia)

→

2

Atur Parameter

Set jumlah rekomendasi, bobot similarity α, filter genre

→

3

Klik Recommend

Sistem menghitung cosine similarity secara on-demand

→

4

Lihat Hasil

Top-N film muncul dengan skor, genre, rating & similarity bar

📥 Download CSV hasil rekomendasi

🔍 Fuzzy search judul film

⚙️ Slider bobot similarity α

🎭 Filter genre real-time

📖 Toggle sinopsis film

🎬 Demo: Aplikasi Streamlit CineMatch

NALAR

🎬 CineMatch App (hosting Cloud)

NALAR

✅ Kesimpulan

📌 Ringkasan

Sistem rekomendasi film berbasis Content-Based Filtering berhasil dibangun menggunakan TF-IDF + Cosine Similarity dengan Bayesian quality scoring, menghasilkan Precision@10 = 0.86 dari 18,139 film TMDB.

✅ Kelebihan

- Tidak membutuhkan data historis user (cold-start ready)
- Interpretable: hasil dapat dijelaskan berdasarkan konten
- Scalable: on-demand cosine similarity efisien
- UI interaktif dengan filter genre & adjustable weight
- Precision@10 tinggi: 86% relevansi genre

⚠️ Keterbatasan

- Hanya berbasis konten, tidak mempelajari selera user
- Cold-start untuk film baru tanpa metadata lengkap
- Evaluasi terbatas pada genre overlap (bukan user feedback)
- Tidak ada personalisasi berbasis riwayat tontonan

🎯 Sistem ini terbukti efektif sebagai baseline Content-Based Recommender dan siap dikembangkan menjadi sistem hybrid yang lebih powerful.

NALAR

🙏

Terima Kasih!

Movie Recommendation System ~ Nalar Bootcamp Final Project

Wa Ode Nisrina Sayyidah Hidayat

Content-Based Movie Recommender · TF-IDF + Cosine Similarity

Nalar Bootcamp · 2026